

项目设计文档

Mapping SQL Agent：面向数仓研发的 SQL 生成、评审与优化工作台



项目定位	本地智能体原型 / 数据研发辅助工具
访问入口	http://127.0.0.1:8000
核心链路	Requirement + Mapping + Skill + Schema -> SQL + Review
文档版本	项目设计说明（日期：2026-05-26）

设计定位

系统以 Mapping 文档为核心输入，结合规则引擎、智能体增强、业务 Skill、Memory 和表结构分析，形成从 SQL 生成、规范校验、版本追踪到 SQL 拆解优化的闭环能力。

1. 项目概述

Mapping SQL Agent 是一个面向数据研发场景的本地智能体原型。系统围绕数据开发过程中最常见的 Mapping 文档、业务需求、业务 Skill、表结构说明和 SQL 规范评审展开设计，把原本分散在文档、脚本、人工经验和代码评审里的工作集中到一个本地工作台完成。它不是单纯的 SQL 生成器，而是一个覆盖 SQL 生成、结果校验、版本追踪、SQL 拆解优化、Skill 管理和表结构理解的完整研发辅助系统。

当前版本定位为本地原型系统，重点验证智能体流程和业务交互闭环。系统不直接连接真实数据库，不执行建表或跑数，而是读取用户上传或粘贴的 Mapping、SQL、DDL、Excel 等文档型输入，完成结构化解析和智能分析。这种设计更贴合企业数据研发的真实前置环节，因为在正式开发 SQL 之前，开发人员通常首先拿到的是需求说明、Mapping 表、表结构文档和历史 SQL，而不是直接操作生产数据。

项目亮点

本项目的亮点在于把规则引擎的稳定性、大模型的语义理解能力、业务 Skill 的场景经验、表结构分析的字段理解能力和版本对比的评审能力组合在一起，形成了一个贴近真实数据研发流程的闭环系统。评委可以从页面中直观看到：输入、生成、校验、对比、优化和分析都已经形成完整链路。

1.1 可访问链接与启动方式

系统以本地 Web 应用形式运行，默认访问地址为 `http://127.0.0.1:8000`。运行时可以通过命令 `python3 webapp.py --host 127.0.0.1 --port 8000` 启动，也可以在 macOS 上通过 `start_webapp.command` 启动。

1.2 主要用户与使用场景

系统面向的核心用户是数据开发人员和 SQL 评审人员。在真实业务中，数据开发人员经常需要根据产品或分析师给出的 Mapping 表手写 SQL，同时还要确认字段映射、Join 关系、过滤条件、聚合口径、分区条件和目标表字段是否一致。这个过程重复性强，而且很容易因为字段理解偏差或历史口径不一致产生问题。

本项目把这些真实痛点分解成五个主要工作区：SQL 生成用于从 Mapping 到 SQL 的初稿生成，版本对比用于需求变更后的差异追踪，SQL 拆解优化用于已有 SQL 的 Review 和优化，Skill 管理用于沉淀托管、清算、核算等业务经验，表结构分析用于理解字段含义与业务用途。五个模块不是孤立功能，而是围绕数据研发生命周期串联起来，能够覆盖从需求输入、经验沉淀到评审优化的主要环节。

1.3 完整性与业务贴合度设计

为了使系统具备完整项目形态，而不是停留在单点生成能力，设计中保留了多类真实研发要素。首先，输入不是单一文本框，而是覆盖 Mapping、Excel、SQL 文件、DDL 表结构、业务需求和可维护 Skill。其次，处理过程不是直接调用模型生成答案，而是先由规则引擎生成可解释草稿，再按需要注入业务 Skill、Memory 和表结构分析结果。最后，输出也不仅是 SQL 文本，还包括规范校验、字段检查、版本记录、差异分析、优化建议和 Skill 复用结果。这个闭环能够体现项目已经覆盖“开发、检查、评审、优化、沉淀”多个环节。

项目还通过内置样例模拟真实数仓业务，当前版本重点围绕托管清算业务口径展开，例如产品清算汇总、托管清算对账、清算失败原因分析、清算 KPI 统计、托管清算对账 Skill、托管估值核算 Skill、资金划拨核验 Skill 和托管清算明细表结构分析等。这些样例不是随意构造的字符串，而是围绕清算流水、交易流水、产品维度、账户维度、清算状态、失败原因、差异金额、分区字段和可复用业务经验设计，能够对应真实托管清核算场景中的清算汇总、金额核验、失败归因和指标复核需求。同时，这套设计并不局限于托管清算部门，后续也可以按相同方式扩展到估值核算、资金划拨、余额对账、交易运营、风险监控和经营分析等其他业务部门场景。

2. 总体设计

总体设计部分用于说明系统整体架构、前后端分工、智能体处理流程和页面组织方式。该项目采用轻量级本地 Web 架构，但在模块划分上按照真实数据研发工作台进行设计，使输入解析、规则生成、模型增强、结果校验、版本沉淀和页面展示形成清晰边界。

系统采用轻量级前后端一体架构。前端由原生 HTML、CSS 和 JavaScript 实现，后端使用 Python 标准库 ThreadingHTTPServer 提供静态页面和 API 服务，核心能力拆分到 src 目录下的多个服务模块。这样的架构便于本地运行、功能验证和快速迭代，也能清晰体现各个智能体能力模块之间的边界。

从工程结构看，webapp.py 承担服务入口和 API 编排职责，web/index.html 与 web/assets/app.js 负责页面结构和交互状态，src 目录下的 agent、sql_generator、sql_style、version_store、sql_insight、schema_insight 等模块负责具体业务能力。config 目录保存 SQL 规范、业务 Skill/Memory 和模型配置，examples 与内置样例用于验证输入输出链路。

模块	主要文件	职责
----	------	----

前端工作台	web/index.html, web/assets/app.js, web/assets/styles.css	提供四个工作区、样例加载、文件上传、状态渲染、复制 SQL、SSE 步骤展示。
Web API 层	webapp.py	提供生成、解析、版本、对比、SQL Insight、Schema Insight、模板下载等接口。
智能体编排	src/agent.py	串联 Mapping 解析、规则 SQL 草稿、智能体增强、规范校验和结果包装。
规则 SQL 生成	src/sql_generator.py	按 WITH、INSERT OVERWRITE、SELECT、JOIN、WHERE、GROUP BY 结构生成 SQL。
规范校验	src/sql_style.py, config/sql_rules.json	检查必需结构块、命名、SELECT *、分号、聚合 GROUP BY、字段覆盖和来源别名。
业务记忆	src/business_memory.py, config/business_memory.json	提供 Skill 选择器和 Memory 条目，增强业务场景理解。
版本存储	src/version_store.py, storage/versions	保存生成版本，支持任务列表、历史版本详情和 SQL/Mapping 差异。
SQL/表结构分析	src/sql_insight.py, src/schema_insight.py	拆解 SQL、给出优化建议；解析 DDL/CSV/JSON/Excel 表结构并推荐 Skill。

2.1 智能体处理流程

2.1.1 输入接收与上下文组织

用户在页面输入 Mapping、SQL 或表结构，也可以通过“加载样例”快速填充演示数据。前端不会只提交单一文本，而是根据当前工作区组织完整上下文，例如生成模式、业务需求、Skill 选择、是否启用表结构分析、上传文件内容和当前版本信息。这样后端收到的请求天然带有业务语义，而不是孤立的 SQL 或 JSON 字符串。

2.1.2 规则引擎兜底与结构化生成

Mapping 输入首先经过本地解析器转换为统一结构，随后由 SQLGenerator 生成规则草稿。规则引擎负责稳定地产生 WITH、INSERT OVERWRITE、SELECT、JOIN、WHERE、GROUP BY 等 SQL 骨架，这一层是系统稳定性的基础。即使大模型调用失败，系统仍然可以用规则结果完成基本 SQL 生成和校验，避免原型演示或实际使用时完全依赖外部模型。

2.1.3 大模型增强与业务记忆注入

在智能体增强模式下，PromptBuilder 会把用户需求、Mapping、规则草稿 SQL、选中的业务 Skill、Memory 条目和表结构分析结果组合为提示词。Skill 用于表达业务场景，当前默认 Skill 已从泛化的同比、环比、用户分层等展示口径，调整为更贴合托管清核算业务的产品清算汇总、托管清算对账、托管估值核算、资金划拨核验、账户余额对账、份额登记确认、清算失败原因分析和清算 KPI 统计；Memory 用于沉淀通用研发经验，例如分区过滤优先、金额口径可追溯、状态字段可复核等。通过这种方式，系统不让模型自由猜测业务含义，而是把当前业务部门真正关心的清算口径、核验规则和字段约束显式注入生成链路中。

2.1.4 结果回传、校验和版本沉淀

后端返回的结果不仅包含 SQL 文本，还包含任务摘要、规范校验、字段覆盖检查、Mapping 诊断、修复后 Mapping、增强上下文和表结构增强结果。每次成功生成 SQL 后，VersionStore 会把 Mapping、SQL、生成模式和需求说明保存为本地版本文件，为后续版本对比提供数据基础。这使系统从一次性生成工具升级为具有追踪能力的研发工作台。

2.2 项目页面整体介绍

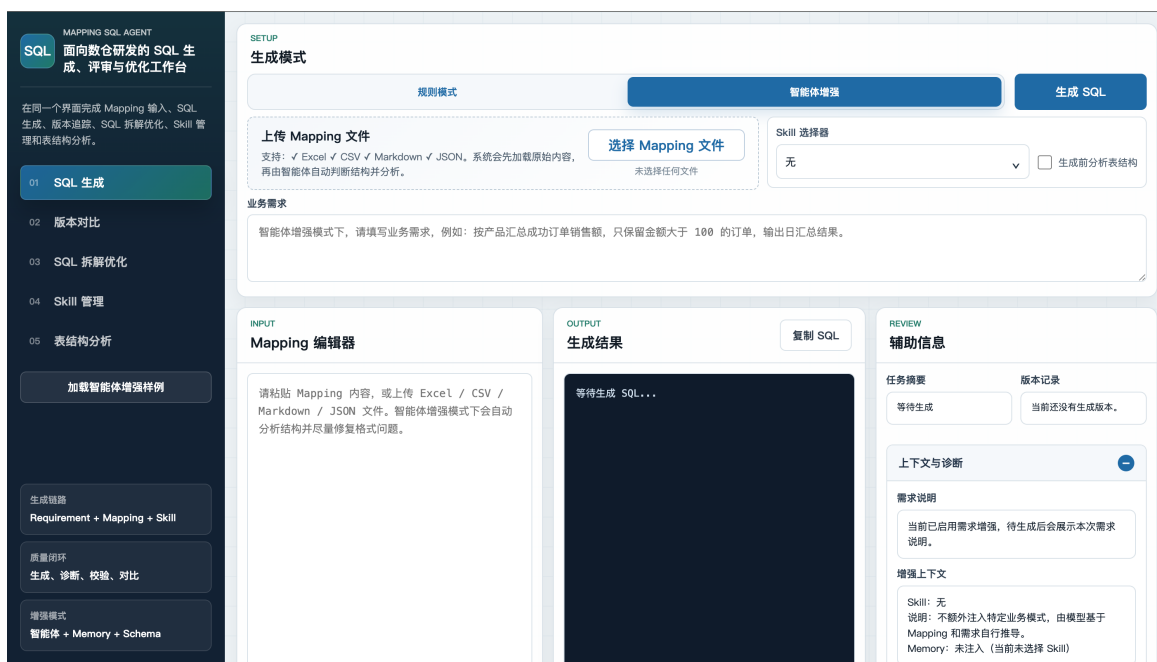


图 1: Mapping SQL Agent 工作台整体页面，左侧为功能导航和系统定位信息，右侧为当前模块的输入、生成与分析区域

从页面组织方式看，Mapping SQL Agent 按照数据研发人员的真实工作路径设计成一个完整工作台。用户进入系统后，首先能在左侧看到 SQL 生成、版本对比、SQL 拆解优

化、Skill 管理和表结构分析五个核心工作区，分别对应“从 Mapping 生成 SQL”“需求变更后比较版本”“评审和优化已有 SQL”“沉淀和维护业务经验”“理解表结构和字段含义”五类高频场景。这样的导航结构能够帮助评委快速判断系统覆盖的业务范围，也能让演示过程按照研发流程自然展开。

页面左侧除了功能导航，还保留了生成链路、质量闭环和增强模式等定位信息，用来说明系统的设计重点。生成链路强调 Requirement、Mapping 和 Skill 的组合输入，质量闭环强调生成、诊断、校验和对比的连续过程，增强模式则说明系统在模型服务、Memory 和 Schema 的辅助下具备业务上下文理解能力。这些信息让页面本身就能传达项目不是一个单点 SQL 拼接工具，而是围绕数仓研发流程搭建的智能辅助系统。

右侧主工作区会随当前模块切换而变化，但整体交互逻辑保持一致：上方通常用于选择模式、加载样例和配置增强能力，中间用于上传或编辑 Mapping、SQL、DDL、Skill 等输入材料，右侧或下方用于展示生成结果、分析结论和辅助诊断。用户在同一个页面内即可完成样例加载、文件上传、需求补充、Skill 选择、Skill 编辑、表结构分析、结果复制和版本查看，减少了在多个工具之间来回切换的成本，也更符合真实数据开发中一边理解需求、一边生成和校验 SQL 的工作方式。

3. 详细设计

详细设计部分围绕五个核心功能模块展开，分别说明 SQL 生成、版本对比、SQL 拆解优化、Skill 管理和表结构分析的输入内容、处理逻辑、输出结果及样例设计。每个模块都对应数据研发流程中的一个实际环节，既可以独立使用，也可以通过样例和上下文信息相互衔接。

3.1 SQL 生成

SQL 生成模块是系统最核心的主流程入口，负责把业务侧提供的 Mapping 文档、研发人员补充的需求说明以及可选的表结构上下文转换为可执行的 SQL 初稿。为了同时兼顾稳定性和智能化能力，系统在这一部分设计了两种生成路径：模板生成模式面向格式规范、规则明确的标准研发场景，智能体增强生成模式面向需求表达更复杂、输入材料更贴近真实业务文档的场景。两种模式并不是互相替代的关系，而是共同覆盖从“标准模板快速出稿”到“复杂需求智能增强”的完整使用链路。

3.1.1 模板生成模式

模板生成模式以本地规则引擎 SQLGenerator 为核心，适合输入已经整理成标准 Mapping JSON 或 Excel Mapping 模板的场景。用户可以直接粘贴标准 Mapping，也可以

下载系统提供的 Excel 模板，在表格中维护目标表、来源表、字段映射、过滤条件、关联关系、聚合规则和分区字段，再上传到系统中解析。系统会把 Excel 内容转换为统一的 Mapping JSON，并按照 target_table、target_partition、sources、joins、filters、target_columns 等结构化字段生成 WITH、INSERT OVERWRITE、SELECT、JOIN、WHERE、GROUP BY 等 SQL 片段。

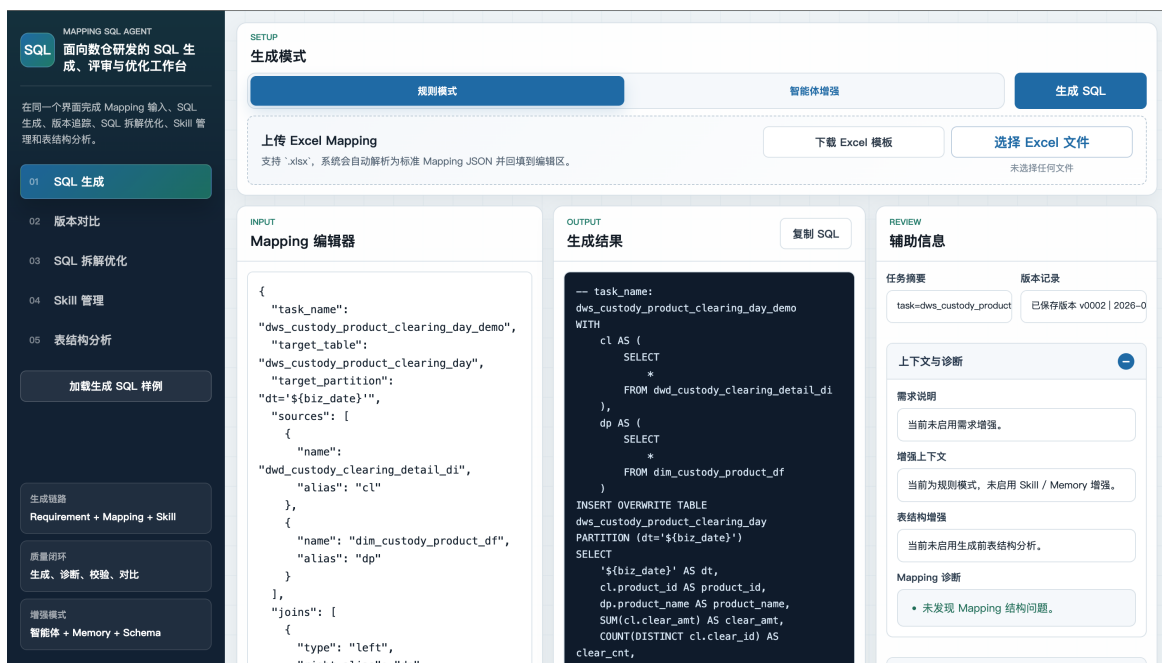


图 2：模板生成模式，加载标准 Mapping 样例后展示标准 Mapping 编辑区与规则生成入口

这种模式的优势在于结果稳定、过程可解释、对外部模型没有强依赖。对于真实数仓开发中大量格式相似的清算汇总、明细加工、对账复核任务，模板生成可以快速给出 SQL 初稿，并且能够清楚地追溯每个字段来自 Mapping 中的哪一项配置。系统内置的“生成 SQL 样例”会加载托管产品清算汇总 Mapping，样例中包含托管清算明细事实表和产品维表，覆盖产品维度、清算金额、清算笔数、失败笔数、清算状态过滤和分区写入等典型内容。点击“加载生成 SQL 样例”后，用户可以看到系统如何把这些结构化配置稳定转换为标准 SQL。

3.1.2 智能体增强生成模式

智能体增强生成模式是在模板生成能力之上的业务增强层。它不是让大模型脱离工程约束从零写 SQL，而是先由本地规则引擎生成可解释的规则草稿，再把用户需求、原始 Mapping、规则草稿 SQL、Skill、Memory 和表结构分析结果一起组织成 Prompt，由模型在明确上下文中进行补全、修正和优化。这样的设计保留了规则生成的稳定底座，同时引

入了模型对自然语言需求、非标准材料和业务经验的理解能力，更接近真实企业环境下数据开发人员与业务方反复沟通后生成 SQL 的过程。

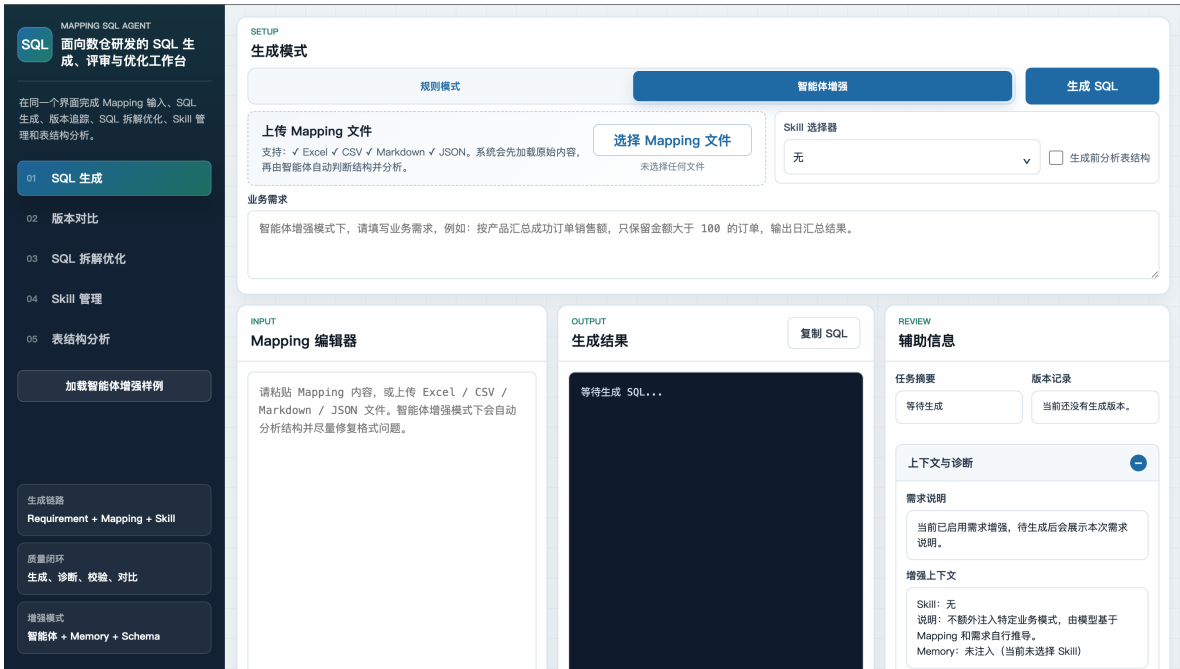


图 3：智能体增强生成模式，样例加载后展示需求、Mapping 编辑与生成结果区域

在用户需求表达方面，增强模式允许用户直接输入更接近业务沟通语言的说明，例如“按产品和日期统计支付成功订单，输出订单数、买家数和支付金额，并只保留指定日期分区”。这类需求如果完全依赖模板字段表达，往往需要提前把所有条件拆成固定配置；增强模式则可以把自然语言需求作为额外约束注入生成过程，让 SQL 不只机械地映射字段，还能体现统计口径、过滤范围、聚合粒度和业务目标。系统在生成 SQL 时会完成 Mapping 理解、规则草稿生成、业务上下文注入、模型增强和结果校验的组合流程，而不是简单拼接字符串。



图 4：需求输入模块，用户可以用自然语言补充统计口径、过滤条件和输出要求

在输入材料方面，增强模式对 Mapping 格式的适应性更强。标准模板仍然可以使用，但系统也允许上传或粘贴 CSV、Markdown、JSON、TXT 等更贴近真实工作流的材料。当业务人员或数据研发同学手里只有半结构化字段说明、临时整理的表格内容、Markdown 文档或文本版映射关系时，系统可以先把原始内容加载到编辑区，再由智能体模型辅助理解字段、来源表和目标字段关系，必要时修复为内部可使用的 Mapping 结构。这个设计解

决了真实场景中“资料不是一开始就完全标准化”的问题，也让系统从演示型工具更接近可落地的研发辅助工作台。

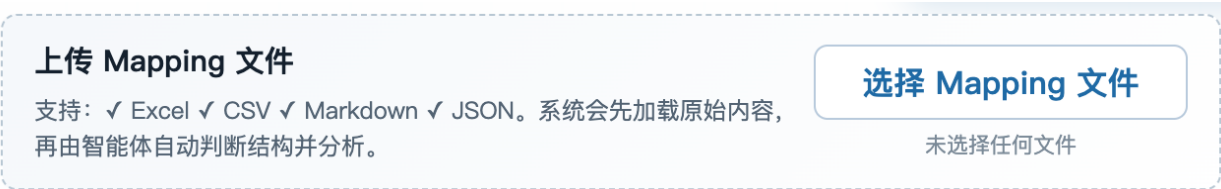


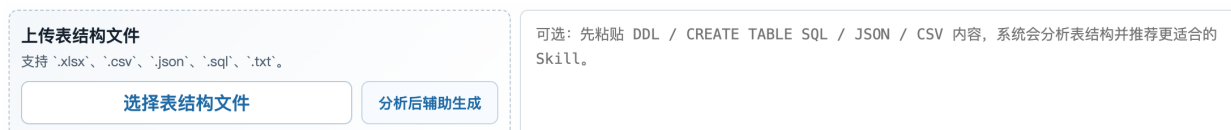
图 5：增强模式 Mapping 上传模块，支持 Excel、CSV、Markdown、JSON 和 TXT 等更丰富的输入格式

Skill 功能是增强模式的重要亮点，主要面向真实场景下的业务复用。传统 SQL 生成工具通常只关注字段和表，却很难表达企业内部长期积累的业务写法。当前版本将 Skill 重点调整为托管清核算业务口径，例如产品清算汇总、托管清算对账、托管估值核算、资金划拨核验、账户余额对账、份额登记确认、清算失败原因分析和清算 KPI 统计等场景化能力。用户在生成前选择相应业务模式后，PromptBuilder 会把该 Skill 的说明、适用场景、生成偏好和业务规则注入模型上下文。这样生成结果就不只是“能跑的 SQL”，而是更容易符合托管清算、核算复核、金额差异定位和失败原因分析等实际业务口径。后续如果切换到其他业务部门，也可以用同样的 Skill 管理方式沉淀对应部门的指标口径和生成策略。



图 6：Skill 选择器，用户可按托管清核算业务场景选择产品清算汇总、托管清算对账、托管估值核算、资金划拨核验、账户余额对账、份额登记确认、清算失败原因分析和清算 KPI 统计等生成策略

表结构分析辅助是另一项贴近实际开发的增强能力。真实数仓环境中，研发人员在写 SQL 之前通常需要先理解表用途、主键候选、Join Key、时间字段、分区字段、指标字段和维度字段，否则很容易出现字段选错、Join 粒度不一致或分区条件遗漏的问题。系统在增强模式中加入“生成前分析表结构”选项，用户可以上传或粘贴表结构文件，系统先分析表用途和关键字段，再把这些结果作为上下文注入 SQL 生成。这样模型在生成 SQL 时能参考表的业务含义和字段角色，后续在 SQL 拆解优化模块中也能继续复用这些表结构理解，形成从生成到评审再到优化的闭环。



上传表结构文件

支持 '.xlsx'、'.csv'、'.json'、'.sql'、'.txt'。

[选择表结构文件](#) [分析后辅助生成](#)

可选：先粘贴 DDL / CREATE TABLE SQL / JSON / CSV 内容，系统会分析表结构并推荐更适合的 Skill。

图 7：生成前表结构分析模块，支持上传表结构文件或粘贴结构文本，为 SQL 生成提供字段理解上下文

在样例演示中，切换到智能体增强模式后再次点击“加载生成 SQL 样例”，系统会同时加载用户需求、示例 Mapping、Skill 选择器和生成前表结构分析配置。界面中的需求输入区展示业务目标，Mapping 编辑区展示更完整的字段映射，Skill 区用于选择业务增强策略，表结构辅助区用于启用生成前分析。点击“生成 SQL”后，等待阶段可以重点介绍系统正在把需求、Mapping、Skill、Memory 和 Schema Insight 合并为增强上下文，并通过智能体模型对规则草稿进行业务化改写。生成完成后，用户不仅能看到 SQL，还能在辅助信息中检查需求说明、增强上下文、表结构增强、Mapping 诊断和字段校验，从而判断生成结果是否真正符合业务场景。



图 8：智能体增强生成样例加载后的整体界面，展示需求、Skill、表结构辅助与 Mapping 的组合输入

3.1.3 规则校验与生成后复核

SQL 生成模块在输出 SQL 初稿后，会继续进入规则校验与字段复核流程。该流程的作用不是简单提示“生成成功”，而是从工程规范角度检查生成结果是否具备可进入评审的基础质量。系统会结合 config/sql_rules.json 中定义的规则，由 SQLStyleChecker 对 SQL 结构进行检查，重点关注是否包含 WITH 或 INSERT 结构、是否正确写入目标表和分区、主查询是否避免 SELECT *、聚合场景是否补齐 GROUP BY、字段别名和来源别名是否清晰，以及生成字段是否覆盖 Mapping 中声明的 target_columns。

对于模板生成模式，规则校验可以验证本地规则引擎生成的 SQL 是否完整反映 Mapping 配置；对于智能体增强模式，规则校验则承担对模型输出进行边界约束的作用。模型可以根据需求、Skill 和表结构上下文进行增强改写，但最终结果仍然要回到本地规则体系中接受检查。这样可以避免模型因为自然语言理解偏差而漏写字段、改变聚合粒度、忽略分区条件或引入不在 Mapping 中的字段，使 AI 生成结果具备可解释、可追溯和可审查的质量保障。



图 9：SQL 生成后的规则校验结果区域，展示生成结果如何经过 Mapping 诊断、SQL 规范检查和字段覆盖复核

前端辅助信息区会展示 Mapping 诊断、规则校验和字段检查等结果。评审人员可以根据这些信息判断当前 SQL 是否符合输入 Mapping、是否满足团队 SQL 编写规范，以及是否存在需要人工复核的风险点。该设计使 SQL 生成模块从“生成一段 SQL 文本”扩展为“生成、校验、提示风险、辅助修正”的完整链路，也更符合真实数据研发中先生成初稿、再进入规范检查和代码评审的工作方式。

3.2 版本对比

版本对比模块用于追踪同一任务不同 Mapping 版本对应的 SQL 变化，解决需求迭代后“改了什么、影响哪里”的评审问题。在真实数据开发中，需求通常不是一次性固定的，

字段、维度、过滤条件和指标口径会反复调整。如果只看最终 SQL，评审人员很难快速判断变化是否合理，因此系统引入版本存储和差异分析能力。

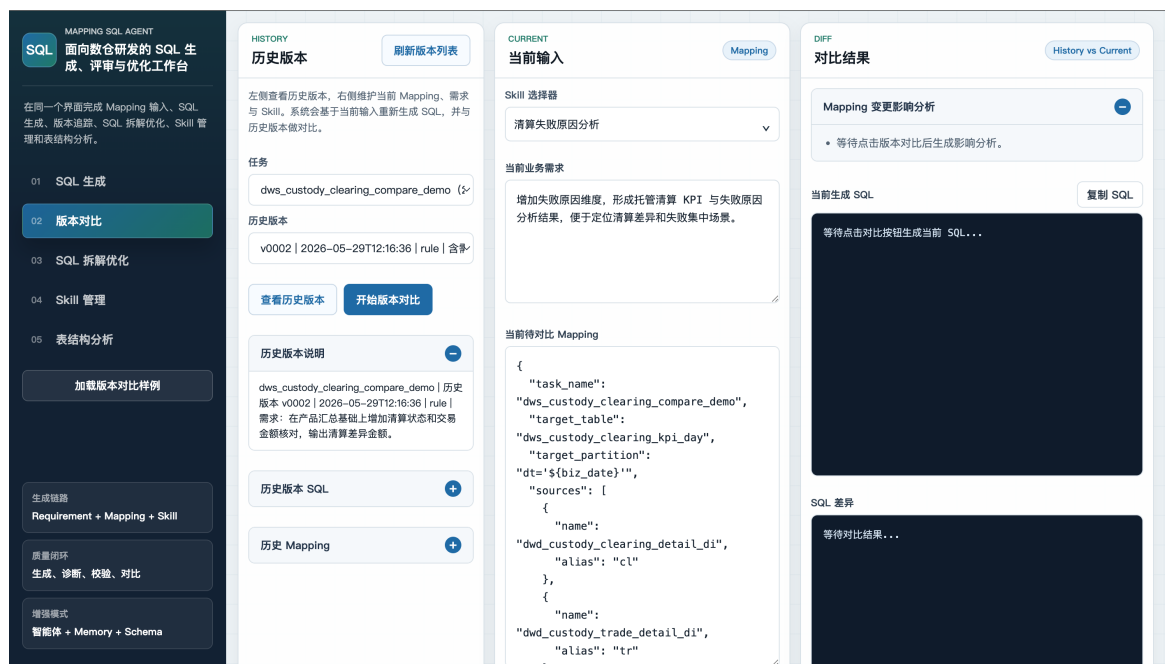


图 10：版本对比工作区，加载历史版本和当前 Mapping 样例

3.2.1 版本沉淀机制

生成成功后，VersionStore 会将 Mapping、SQL、任务摘要、生成模式、规范问题和用户需求说明保存为本地 JSON 版本记录。每个任务按 task_name 建立独立目录，并用 v0001、v0002 等编号持续追加。这个设计模拟了真实研发中的版本留痕，便于后续回看历史口径。

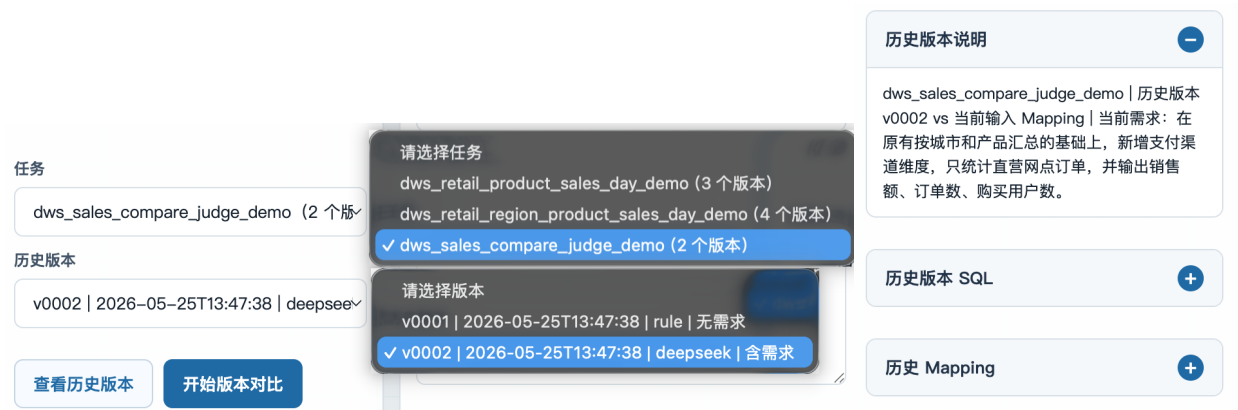


图 11 左：任务与历史版本选择区域；右：历史版本说明区域

3.2.2 差异分析逻辑

用户选择历史版本后，系统会读取历史 Mapping 与历史 SQL，再基于当前输入重新生成一版当前 SQL。随后系统分别对 SQL 文本和 Mapping JSON 做逐行 diff，并通过 MappingImpactAnalyzer 分析字段、来源表、过滤条件和 Join 变化可能带来的影响。这样评审人员既能看到代码层面的差异，也能看到业务映射层面的变化。

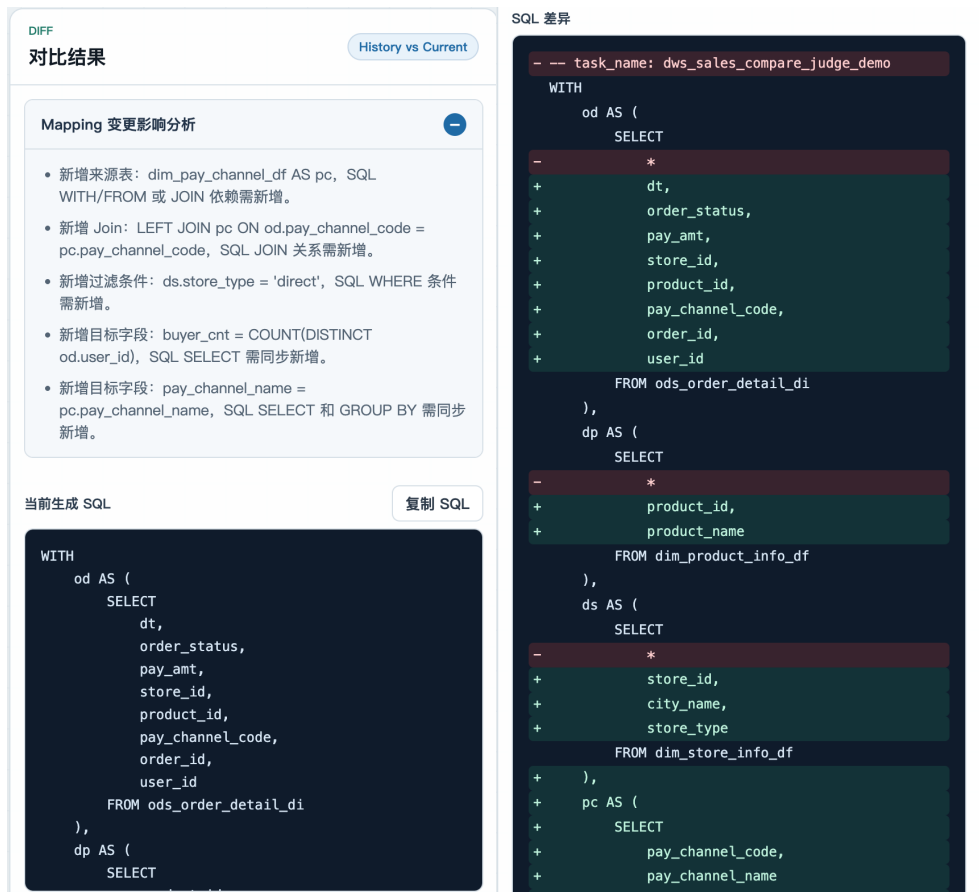


图 12 左：当前生成结果与 Mapping 影响分析；右：SQL 差异高亮对比结果

3.2.3 样例输入设计

版本对比样例使用任务 dws_custody_clearing_compare_demo。历史版本从按产品汇总每日托管清算金额和清算笔数，逐步演进到增加清算状态、交易金额核对和差异金额输出；当前版本继续新增产品维表、失败原因维度、失败笔数和清算 KPI 统计口径。这个样例模拟了真实清算业务迭代中常见的“新增核验维度、补充状态口径、定位失败原因、完善指标字段”的变化过程，能够体现版本对比功能在业务需求变更评审中的实际价值。

3.3 SQL 拆解优化

SQL 拆解优化模块面向已有 SQL 的审查与优化场景。很多真实项目中并不是从零开始写 SQL，而是需要阅读历史 SQL、接手他人 SQL 或对已有 SQL 做评审。该模块允许用户上传或粘贴 SQL，并结合 Skill 和表结构上下文进行语义分析与优化建议生成。



图 13: SQL 拆解优化工作区，样例 SQL、Skill 和表结构样例已加载

3.3.1 SQL 结构理解

SQLInsightService 会对输入 SQL 进行结构拆解，识别 SELECT、FROM、JOIN、WHERE、GROUP BY、ORDER BY、WINDOW 等片段，并提取来源表、Join 数量和目标输出表。即使不调用大模型，系统也能通过本地规则给出基础的结构说明和优化建议，例如避免 SELECT *、检查过滤条件是否可以前置、确认 GROUP BY 字段是否冗余等。

3.3.2 结合 Skill 与表结构优化

当启用增强分析时，SQL 拆解模块会将 SQL 文本、业务 Skill、Memory 和可选表结构分析结果一起交给模型。以托管清算对账场景为例，系统会知道当前 SQL 更关注产品、清算状态、清算金额、交易金额、差异金额、失败笔数和失败原因等字段，并可以结合表结构判断哪些字段是指标字段、维度字段、状态字段或分区字段。这样输出的优化建议不只是语法层面的，也更贴近清算复核、金额核验和异常定位的真实业务要求。

3.3.3 样例输入设计

该模块内置托管清算对账 SQL，加载样例时会自动选择“托管清算对账” Skill，并勾选生成前表结构分析，同时填入托管清算明细表 DDL 样例。这个设计用于展示 SQL 文本、业务场景和表结构信息如何共同参与 SQL Review，使系统看起来更接近真实数据研发中的审查流程。

3.4 Skill 管理

Skill 管理模块用于把原本写死在系统中的业务 Skill 扩展为可查看、可编辑、可创建、可沉淀的业务经验库。早期系统只有同比汇总、环比汇总、产品维度汇总、用户分层和 KPI 统计等通用 Skill，能够满足演示中的常见分析场景，但与托管、清算、核算等更具体的业务场景仍有距离。新增 Skill 管理后，用户可以在页面中查看已有 Skill，编辑名称、适用场景、SQL 生成偏好、示例场景和业务规则，也可以创建新的托管清核算类 Skill，使智能体上下文从固定配置升级为可维护的业务知识资产。



图 14: Skill 管理工作区整体页面，展示已有 Skill、Skill 编辑器和 Skill 生成器的组合布局

3.4.1 已有 Skill 查看与编辑

已有 Skill 列表只展示 Skill 名称和英文 ID，避免列表区域过于拥挤；具体说明、SQL 生成偏好、示例场景和业务规则放在右侧编辑区展示。用户点击某个 Skill 后，系统会把该 Skill 的结构化内容回填到编辑器中，支持继续修改和保存。保存后的 Skill 会写回本地业务记忆配置，并同步刷新 SQL 生成、版本对比和 SQL 拆解优化模块中的

Skill 选择器。这样可以保证业务经验不是只停留在文档或口头说明中，而是能够真正参与后续智能体生成和优化。

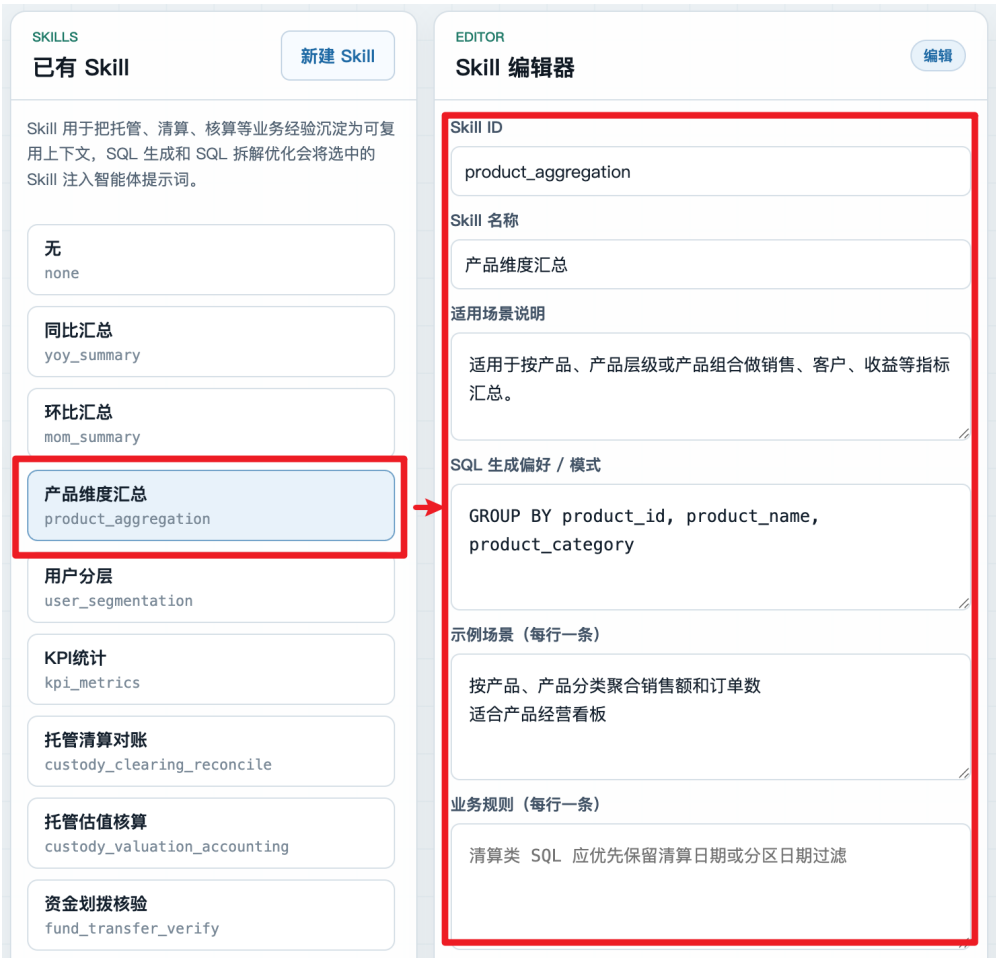


图 15：已有 Skill 查看与编辑区域，展示 Skill 名称、英文 ID 和结构化编辑内容

3.4.2 Skill 生成器设计

Skill 生成器面向不知道如何编写 Skill 的用户。用户只需要输入业务场景、可选的表结构或 Mapping 材料，以及生成要求，系统就会生成一个结构化 Skill 草稿。当前实现重点面向托管清核算业务，能够根据托管清算、估值核算、资金划拨、份额登记、余额核对等关键词推导 Skill 名称、适用说明、SQL 生成偏好、示例场景和业务规则。例如托管清算对账 Skill 会强调产品、账户、交易日期、清算日期、状态、清算金额、失败笔数和差异金额等字段角色，并要求优先保留分区日期或清算日期过滤。

EDITOR

Skill 编辑器

草稿

Skill ID

custody_clearing_reconcile_6940

Skill 名称

托管清算对账

适用场景说明

适用于托管清算对账场景下的 SQL 生成、拆解和优化，强调业务口径、字段角色、分区过滤和结果可复核。典型需求：要求优先使用 dt 或 clear_dt 过滤，金额字段使用 SUM，状态

SQL 生成偏好 / 模式

按产品、账户、交易日期、清算日期、状态分组；输出金额、笔数、成功笔数、失败笔数、差异金额等指标；优先使用分区日期、清算日期、交易日期、状态过滤

示例场景（每行一条）

按产品、账户、交易日期、清算日期、状态统计金额、笔数、成功笔数、失败笔数、差异金额
对托管清算对账结果进行字段覆盖、分区过滤和异常状态复核

业务规则（每行一条）

生成 SQL 时应优先保留业务日期或分区日期条件，避免全表扫描。
金额、份额、净值等指标字段应明确聚合方式，并注意空值处理。

保存 Skill

删除 Skill

FACTORY

Skill 生成器

托管清算核算

业务场景

托管清算场景下，按产品、交易日期、清算日期和清算状态统计清算金额、成功笔数、失败笔数和差异金额，并识别清算失败原因。

可选：表结构 / Mapping / 样例 SQL

CREATE TABLE
dwd_custody_clearing_detail_di (
product_id STRING COMMENT '产品编号',
account_id STRING COMMENT '托管账户'

可选：生成要求

要求优先使用 dt 或 clear_dt 过滤，金额字段使用 SUM，状态字段需要区分成功、失败和处理中，输出结果要便于对账复核。

生成 Skill 草稿

加载托管清算样例

草稿预览

{
"id": "custody_clearing_reconcile_6940",
"name": "托管清算对账",
"description": "适用于托管清算对账场景下的 SQL 生成、拆解和优化，强调业务口径、字段角色、分区过滤和结果可复核。典型需求：要求优先使用 dt 或 clear_dt 过滤，金额字段使用 SUM，状态字段需要区分成功、失败和处理中，输出结果要便于对账复核。",

图 16: Skill 生成器草稿生成结果，展示托管清算对账场景如何被整理为结构化 Skill

3.4.3 与智能体上下文的关系

Skill 管理模块的价值在于强化系统对 AI 的约束能力。新增或编辑后的 Skill 会被 PromptBuilder 注入 selected_skill 区域，其中包含 Skill 名称、描述、SQL 模式、示例场景和业务规则。模型在生成 SQL 或分析 SQL 时，不再只依赖用户临时输入和 Mapping 字段，而是可以参考已经沉淀的业务经验。这样做可以降低模型自由发挥的空间，让托管清算、估值核算等场景的统计口径、分区习惯、金额处理和状态识别规则以结构化方式进入上下文，从而形成“业务经验沉淀—Skill 注入—生成约束—规则校验”的闭环。

3.5 表结构分析

表结构分析模块用于将 DDL、Excel、CSV、JSON 或文本形式的表结构转换成结构化元数据理解结果。系统不会执行 CREATE TABLE，也不会创建数据库表，只把 DDL 当作表结构文档解析。这个设计符合真实企业环境中先读取元数据、再辅助 SQL 开发的流程。

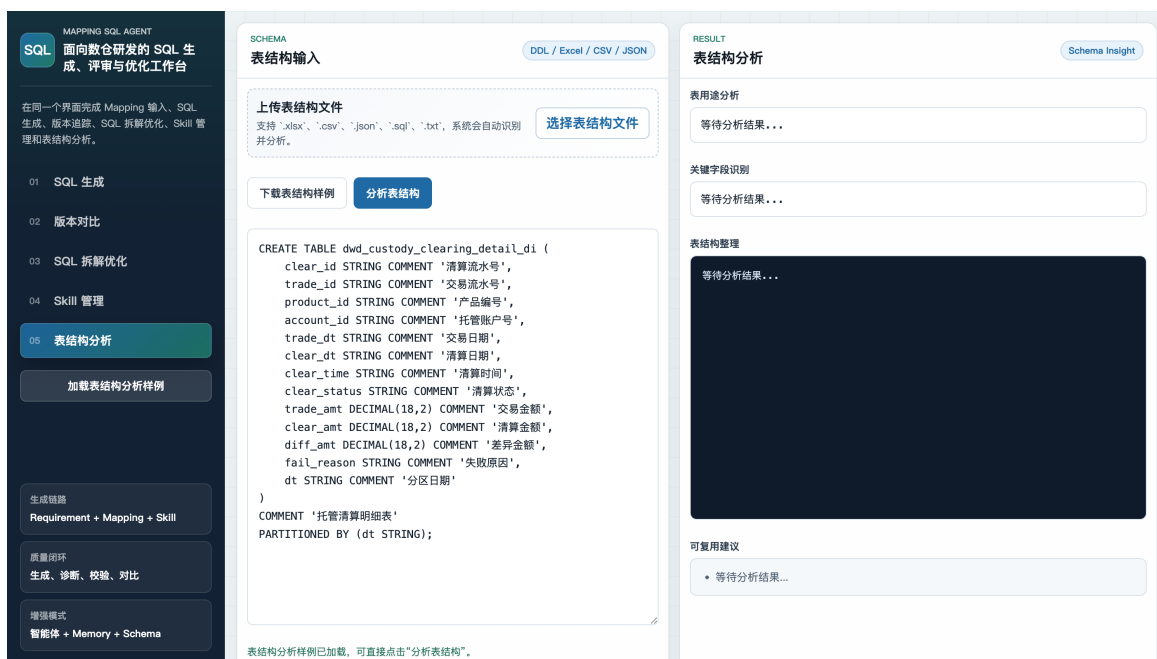


图 17：表结构分析工作区，DDL 样例已解析出用途、关键字段和结构整理

3.5.1 元数据解析能力

Schema Insight Service 会提取字段名、字段类型和字段注释，并基于字段命名和注释推断表用途。系统还会识别主键候选、Join Key、时间字段、分区字段、指标字段和维度字段。这些信息可以反向服务于 SQL 生成和 SQL 拆解优化，使系统对字段的理解不仅停留在字符串层面。

3.5.2 样例输入设计

表结构分析样例使用 `dwd_custody_clearing_detail_di` 的 CREATE TABLE DDL，字段包括清算流水号、交易流水号、产品编号、托管账户号、交易日期、清算日期、清算状态、交易金额、清算金额、差异金额、失败原因和分区日期。该样例模拟真实数仓中的托管清算明细表，能够体现系统如何从 DDL 中理解清算事实表用途、金额指标、产品与账户维度、状态字段、失败原因字段以及 `dt` 分区字段。

3.5.3 输出结果设计

结果区先展示表用途分析和关键字段识别，再展示表结构整理和可复用建议。这样的顺序符合真实阅读习惯：评审人员先理解这张表用于什么业务，再看关键字段和字段列表，最后判断它可以复用于哪些 SQL 生成或分析场景。

4. AI 技术应用方案

AI 技术应用方案重点说明系统如何把大模型能力纳入受控流程。项目没有把模型作为唯一生成器，而是将其放在规则引擎、业务 Skill、表结构分析、提示词约束和输出校验共同组成的智能体链路中使用。底层模型能力可由 DeepSeek API 提供，但系统价值并不等同于简单调用 API，而在于围绕 SQL 研发任务设计了上下文控制、生成边界和质量复核机制。

4.1 AI 应用总体思路

本项目的 AI 应用思路可以概括为“规则先行、上下文筛选、模型增强、规则复核”。规则先行是指系统先用本地规则引擎解析 Mapping 并生成稳定 SQL 草稿；上下文筛选是指只把当前任务真正需要的用户需求、Mapping、Skill、Memory 和表结构分析传给模型，避免无关信息造成上下文污染；模型增强是指让模型在既定骨架中补充自然语言需求理解、非标准 Mapping 修复和业务口径优化；规则复核是指模型输出后仍然进入字段覆盖、别名、聚合、SELECT * 和结构完整性检查。通过这一流程，AI 被用作受约束的业务增强能力，而不是不可控的黑箱生成器。

4.2 输入与文件支持

系统针对不同业务输入设计了不同的文件支持策略。Mapping 输入在规则模式下支持标准 JSON 和 Excel Mapping，目的是保证结构稳定和可解析；在智能体增强模式下，系统进一步允许 CSV、Markdown、JSON、TXT 等原始内容作为输入，由模型辅助理解和修复非标准 Mapping。SQL 拆解优化模块支持 .sql、.txt 和直接粘贴，表结构分析模块支持 DDL、Excel、CSV、JSON、SQL 和 TXT。模板下载能力则用于降低用户构造样例或标准输入文件的成本。

4.3 智能体增强与兜底机制

后端通过模型配置服务从本地 JSON 读取模型配置，底层模型能力可由 DeepSeek API 提供。增强模式下，PromptBuilder 会把业务需求、Mapping、规则草稿、Skill、Memory 和表结构分析组合为提示词。若智能体模型调用失败，SQL 生成会回退到规则引擎，SQL 拆解和表结构分析会回退到本地启发式分析。这种兜底机制是系统完整性的关键，因为真实业务系统不能因为外部模型接口偶发失败就完全不可用。

从设计角度看，规则引擎负责确定性、模型负责语义补充、Skill 和 Memory 负责业务经验注入、表结构分析负责字段理解。这四层能力互相补充，使系统既能展示智能化，又保留工程系统应有的稳定性和可解释性。

4.4 规则约束与质量校验设计

系统的规则约束不只是对生成结果做格式检查，而是贯穿输入、生成和输出复核的完整质量控制链路。输入阶段，系统要求 Mapping 至少包含目标表、分区、来源表、目标字段、字段表达式和必要过滤条件，并将 Excel、JSON 等材料统一解析为内部结构；生成阶段，SQL Generator 先根据结构化 Mapping 生成 WITH、INSERT OVERWRITE、PARTITION、SELECT、FROM、JOIN、WHERE 和 GROUP BY 等可解释 SQL 骨架；输出阶段，SQLStyleChecker 再根据 config/sql_rules.json 对关键结构块、命名规范、SELECT *、结尾分号、聚合 GROUP BY、字段覆盖和来源别名合法性进行复核。

这套设计的核心目的，是把大模型放在受控的工程流程中使用，而不是把需求和 Mapping 一次性丢给模型自由发挥。规则草稿相当于给模型提供了一条确定性的生成基线，模型只能在已有字段来源、目标字段和 SQL 结构的范围内做语义补充和优化。这样既能利用模型理解非标准需求、业务口径和自然语言表达的能力，又能降低模型临时编造字段、忽略分区、漏写目标字段或改变统计粒度的风险。

从上下文控制角度看，系统通过“生成前约束、生成中约束、生成后约束”减少上下文污染。生成前的 Skill、Memory 和表结构分析用于筛选真正需要注入模型的业务信息，避免把无关经验、无关表字段或过多历史上下文混入 Prompt；生成中的规则草稿用于限定模型应参考的 SQL 骨架和字段范围；生成后的规则校验用于把模型输出重新拉回本地规则体系。对于不符合规则的结果，系统能够在校验区暴露问题，为人工修正或后续自动修正提供明确依据。

设计层次	实现位置	约束作用
输入结构约束	src/mapping_loader.py	要求 Mapping 包含任务名、目标表、分区、来源表和目标字段，并校验 sources 与 target_columns 的基本完整性。
规则 SQL 草稿	src/sql_generator.py	按 WITH、INSERT OVERWRITE TABLE、PARTITION、SELECT、FROM、JOIN、WHERE、GROUP BY 生成稳定 SQL 骨架。
规范规则配置	config/sql_rules.json	集中维护必需结构块、命名规范、分号、主查询 SELECT *、聚合 GROUP BY、字段覆盖和来源别名等规则。
质量校验执行	src/sql_style.py	对生成 SQL 做规范校验和字段检查，返回 style_issues、field_checks 和 rule_profile。
智能体增强约束	src/agent.py、 src/prompt_builder.py	先生成规则草稿，再把 Mapping、需求、Skill、Memory、表结构分

		析和规则草稿一起注入 Prompt，避免模型脱离工程约束自由生成。
--	--	-----------------------------------

4.5 智能体提示词与输出约束设计

本项目的智能体增强可以理解为一套面向 SQL 研发的上下文编排机制。系统首先判断模型需要看到哪些内容，再通过提示词明确模型必须遵循哪些规则，最后用本地校验器检查输出是否满足 Mapping 和 SQL 规范。相比直接调用大模型生成 SQL，这种设计更强调“给模型看什么、让模型关注什么、要求模型输出什么、输出后如何复核”，从而把模型能力转化为可解释、可审阅、可回退的工程能力。

在生成前，PromptBuilder 不会只传入一句自然语言需求，而是将用户需求、Mapping 原文、规则引擎生成的 SQL 草稿、选中的 Skill、业务 Memory 和可选的表结构分析结果按固定结构组织。用户需求用于表达统计目标和过滤口径，Mapping 用于限定目标表、来源表、字段映射和关联条件，规则草稿用于提供可追溯的 SQL 骨架，Skill 用于注入当前业务场景的生成策略。当前版本以托管清核算为主要口径，Skill 会重点表达产品清算汇总、托管清算对账、托管估值核算、资金划拨核验、账户余额对账、份额登记确认、清算失败原因分析和清算 KPI 统计等业务规则；Memory 用于提供团队沉淀的通用口径，表结构分析用于补充表用途、Join Key、分区字段、时间字段、指标字段和维度字段。通过这些输入约束，模型不需要凭空猜测业务含义，也不容易被无关上下文带偏；当系统扩展到其他业务部门时，也可以替换或新增对应部门的 Skill 与 Memory。

在生成中，提示词会明确规定模型应以规则草稿为基础进行增强，不允许脱离 Mapping 自行新增不存在的目标字段或来源字段。系统同时约束 SQL 关键字大写、表名和字段别名小写、优先使用 WITH 组织来源表、主查询不使用 SELECT *、必须覆盖 mapping.target_columns、存在聚合表达式时必须生成 GROUP BY，并要求只返回 SQL，不输出解释性文字和 Markdown 代码块。对于 SQL 拆解和表结构分析场景，提示词还要求返回合法 JSON，保证后端可以继续解析和展示。

在生成后，模型输出不会被直接视为最终答案，而是继续进入 SQLStyleChecker 的规范校验和字段检查流程。校验结果会指出结构缺失、字段覆盖不足、来源别名异常、聚合规则不一致等问题，使系统具备对模型输出进行复核和纠偏的能力。也就是说，模型在本项目中承担语义理解和增强改写职责，规则引擎与校验器承担边界控制和质量把关职责，二者共同形成更接近真实数据研发流程的智能体闭环。

控制环节	给模型看的内容或提出的要求	设计目的
输入上下文	用户需求、Mapping、规则草稿 SQL、Skill、Memory、表结构分析。	让模型理解业务目标、字段来源、团队口径和表结构含义，避免凭空推断。

提示词规则	SQL 关键字大写、别名小写、禁止主查询 SELECT *、覆盖 target_columns、聚合必须 GROUP BY。	把企业 SQL 规范显式写入模型任务，减少自由发挥。
输出格式	SQL 生成场景只返回 SQL；分析场景返回合法 JSON，不输出 Markdown 代码块。	保证后端可以稳定解析、展示和继续校验结果。
结果校验	SQLStyleChecker 检查必需结构块、字段覆盖、来源别名和聚合规则。	用确定性规则复核模型结果，避免模型输出绕过工程约束。
失败兜底	模型调用失败时回退规则 SQL 或本地启发式分析。	保证系统可用性，不让外部模型服务成为单点风险。

4.6 Skill 管理与业务知识沉淀

Skill 管理是 AI 技术应用方案中的业务知识沉淀层。系统不再只依赖内置的几个通用 Skill，而是允许用户将托管清算对账、托管估值核算、资金划拨核验等业务经验维护为结构化 Skill。每个 Skill 都包含名称、适用场景、SQL 生成偏好、示例场景和业务规则，其中业务规则会随 Prompt 一起注入模型上下文。通过这种方式，系统可以控制模型需要了解哪些业务经验，也可以避免把无关经验混入当前任务，进一步加强上下文控制和输出约束。

4.7 样例加载设计

样例加载设计用于保证评审人员在没有真实数据库和生产数据的情况下，也能完整理解系统链路。SQL 生成样例展示 Mapping 到 SQL 的生成过程，版本对比样例展示需求迭代造成的结构变化，SQL 拆解优化样例展示历史 SQL Review 流程，Skill 管理样例展示托管清算业务经验如何被整理为结构化 Skill，表结构分析样例展示 DDL 如何转化为字段角色和可复用建议。样例不是孤立测试数据，而是贯穿功能模块、AI 约束和业务场景的演示输入集合。

入口	样例来源	加载内容与目的
SQL 生成样例	builder_rule / builder_agent_enhanced	规则样例填充产品销售 Mapping；智能体增强样例额外填充业务需求、产品维度 Skill 和表结构辅助。
版本对比样例	COMPARE_SAMPLE	创建两个历史版本和一个当前版本，演示新增维度、过滤条件和字段后 SQL/Mapping 差异。
SQL 分析样例	SQL_INSIGHT_SAMPLE +	填入产品销售 SQL，自动选择产品维度汇总

	SCHEMA_SAMPLE	Skill，并加载表结构 DDL。
表结构分析样例	SCHEMA_SAMPLE	填入账户交易明细表 DDL，演示字段提取、用途分析、关键字段识别和复用建议。

4.8 数据存储与安全边界

生成版本以 JSON 文件形式存放在 storage/versions，便于本地演示和对比。本地模型配置文件用于存放模型配置和 API Key，不应提交到公开仓库。当前系统不连接真实数据库，不执行 SQL，也不创建表；所有内置样例都是文档型输入，用于模拟真实业务中常见的 Mapping、SQL 和 DDL 资料。

如果后续扩展到生产环境，可以在当前设计基础上接入企业元数据平台、数据目录、SQL 执行网关和权限系统。届时表结构可以从真实元数据服务自动读取，SQL 可以提交到测试环境执行，版本对比也可以进一步接入 Git 或代码评审平台。

5. 测试方案

测试方案围绕功能正确性、AI 生成质量、规则校验有效性、异常兜底能力和前端交互完整性展开。由于当前系统定位为本地原型，不连接真实数据库、不执行生产 SQL，因此测试重点不是跑数结果，而是验证输入能否被正确解析、生成链路能否稳定完成、输出能否被规则检查、样例能否覆盖主要业务场景，以及模型异常时系统是否仍然具备可用结果。

5.1 测试目标与测试范围

测试目标是确认系统能够支撑评审材料中描述的完整 workflows。测试范围包括 SQL 生成、版本对比、SQL 拆解优化、Skill 管理、表结构分析五个页面模块，也包括 Mapping 文件上传、SQL 文件上传、表结构文件上传、样例加载、智能体增强、Skill 选择、Skill 草稿生成、Skill 保存编辑、生成前表结构分析、版本存储、规则校验和异常提示等关键交互。测试数据以项目内置样例为主，同时使用空输入、非法 JSON、非标准文本、模型调用失败和 Skill 维护等场景验证鲁棒性。

5.2 功能测试设计

SQL 生成测试主要验证模板生成和智能体增强两条路径。模板生成需要检查标准 JSON Mapping 和 Excel Mapping 是否能被解析为统一结构，并生成包含 WITH、INSERT OVERWRITE、PARTITION、SELECT、JOIN、WHERE 和 GROUP BY 的 SQL 初稿；智能体增强

需要检查需求说明、Mapping、Skill 和表结构分析是否能够共同影响生成结果。版本对比测试需要验证历史版本、当前版本、SQL 差异和 Mapping 影响分析是否能够对应展示。SQL 拆解优化测试需要验证 SQL 作用分析、优化建议、结构拆解和优化后 SQL 的展示顺序。Skill 管理测试需要验证已有 Skill 列表展示、Skill 详情编辑、托管清算样例加载、Skill 草稿生成、保存后同步到各模块选择器等链路。表结构分析测试需要验证 DDL、Excel、CSV、JSON 和文本输入是否能输出表用途、关键字段、结构整理和可复用建议。

5.3 AI 生成质量测试

AI 生成质量测试重点关注模型是否受到上下文控制，而不是仅看是否返回一段 SQL。测试时需要观察模型生成是否遵循规则草稿，是否覆盖 Mapping 中的目标字段，是否正确理解 Skill 代表的业务场景，是否利用表结构分析中的 Join Key、分区字段、指标字段和维度字段，是否避免自行编造不存在的字段。对于 SQL 拆解和表结构分析，还需要检查模型返回内容是否符合后端可解析的 JSON 结构，避免出现解释性文字、Markdown 代码块或不可解析内容。

5.4 规则校验与异常场景测试

规则校验测试围绕 config/sql_rules.json 和 SQLStyleChecker 展开，重点检查必需结构块、主查询 SELECT *、字段覆盖、来源别名、聚合 GROUP BY 和命名规范等规则是否能够发现问题。异常场景测试包括空 Mapping、非法 JSON、缺少目标字段、缺少来源表、上传不支持文件类型、模型服务不可用和重复点击提交等情况。系统应在这些场景下给出明确提示或回退到本地规则引擎，不能出现页面无响应、结果区域混乱或用户无法判断失败原因的情况。

5.5 前端交互与样例加载测试

前端交互测试主要验证五个页面工作区的切换、按钮状态、加载提示、文件名展示、结果刷新和复制操作是否正常。样例加载测试需要分别点击 SQL 生成样例、版本对比样例、SQL 分析样例、Skill 管理样例和表结构分析样例，确认样例内容不仅填入主输入框，也能同步带入 Skill 选择、Skill 草稿、生成前表结构分析、历史版本、当前版本和 DDL 表结构等辅助信息。这样可以保证评委在审阅材料时能够直接看到系统完整链路，而不是只能看到孤立文本。

5.6 测试结论与改进方向

从当前原型测试目标看，系统已经覆盖核心功能链路、典型样例链路、AI 增强链路和规则校验链路，能够支撑本地演示和评审材料说明。后续如果进入生产化阶段，测试方

案还可以进一步扩展为单元测试、接口测试、端到端浏览器测试和 SQL 试运行测试，并接入真实元数据平台或测试库，对字段血缘、执行计划、性能风险和数据结果一致性进行更严格验证。

6. 经验总结与扩展方案

本章总结项目建设过程中形成的工程经验、边界处理方式和后续扩展方向。项目的核心经验是：智能化系统不能只追求模型回答能力，还需要把输入材料、业务上下文、生成过程、规则校验和异常兜底都纳入工程设计。只有这样，AI 才能真正服务于数据研发流程，而不是停留在一次性问答工具层面。

6.1 工程经验与边界处理

6.1.1 加载状态与重复提交控制

前端在生成、分析和对比过程中会将相关按钮置灰，并显示处理中状态，防止用户重复提交同一个请求。生成和对比接口支持流式步骤返回，页面可以展示当前正在解析 Mapping、推导字段、注入上下文或生成结果，从交互层面提升系统的可感知性。

6.1.2 空输入与格式异常处理

系统在前端和后端都对必填输入进行检查，例如 Mapping 文本、SQL 内容、表结构文本或上传文件。规则模式要求 Mapping 为合法 JSON，以保证结构化生成的确定性；智能体增强模式则允许输入更接近真实业务文档的非标准内容，并尝试通过模型修复 Mapping 格式。

6.1.3 模型失败与本地兜底

当智能体模型调用失败时，SQL 生成会回退到本地规则引擎，SQL 拆解和表结构分析会回退到本地启发式分析，并把 fallback_reason 返回给前端。这种设计避免系统完全依赖外部服务，也体现了工程系统对异常场景的考虑。

6.1.4 敏感配置边界

模型 API Key 仅作为本地配置存放在 config/deepseek_config.json 中；当前原型可通过 DeepSeek API 提供底层模型能力，不应提交到公开仓库。项目设计文档也只说明配置机制，不暴露任何真实密钥。这个边界对于真实项目非常重要，因为模型调用能力属于外部服务接入点，需要和业务代码、样例数据、公开文档保持隔离。

6.2 样例数据的设计经验

内置样例不是生产数据，而是用于验证各模块输入输出契约的最小测试集。样例覆盖 Mapping 生成、版本差异、SQL 拆解、Skill 生成和 DDL 表结构解析五类核心输入，使原型在没有真实数据库的情况下也能完整展示系统链路。

6.3 可扩展方向

后续扩展可以围绕生产化接入继续推进。当前版本已经以托管清算业务口径作为主要样例，证明系统可以把某一业务部门的字段习惯、指标口径、核验规则和评审经验沉淀为 Skill、Memory、表结构分析和规则校验链路。第一步可以接入真实数据库或企业元数据平台，使表结构不再依赖用户手动粘贴 DDL，而是从数据目录自动读取字段、注释、分区和血缘信息。第二步可以增强 SQL 规范配置和代码审查规则，把企业内部的命名规范、分区规范、指标口径和性能规则沉淀为可配置策略。第三步可以将版本对比能力接入 Git 或代码评审平台，使系统不仅能在页面中展示差异，也能辅助 Pull Request 审查。第四步可以持续沉淀更多业务 Skill，把托管清算经验扩展到估值核算、资金划拨、余额对账、交易运营、风险监控、客户经营和管理驾驶舱等其他业务部门场景。